

Lecture 2: Supervised Learning
The Recipe of Deep Learning

Deep Learning for Scientific Discovery

Marcin Płodzień

Atomic Optics Department
Institute of Theoretical Physics
Jagiellonian University in Kraków



1

The Recipe of Deep Learning

JAGIELLONIAN UNIVERSITY
IN KRAKOW

Neural Network Anatomy

under the hood

Non-linear, parametrised functions:

Layers: input $\rightarrow$ hidden $\rightarrow$ output

Each neuron computes

$y = \sigma\left(\sum_i x_i w_i + b\right)$

Layers are nested:

$f_{\vec{\theta}}(\mathbf{x}) = h^{(L)} \circ \cdots \circ h^{(1)}(\mathbf{x})$

$\sigma =$ **activation function**, e.g.:

sigmoid: $\frac{1}{1+e^{-x}}$



tanh: $\frac{e^{x}-e^{-x}}{e^{x}+e^{-x}}$

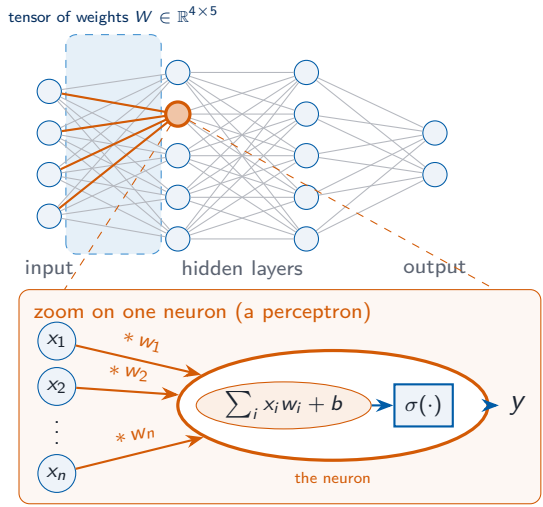


ReLU: $\max(0, x)$

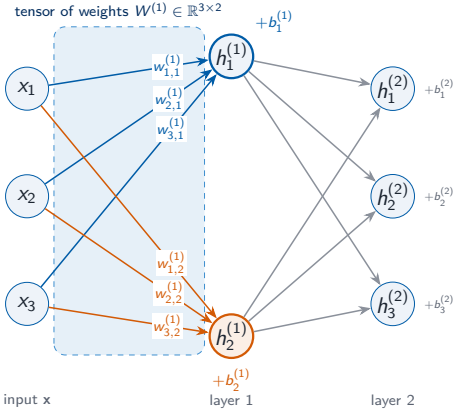


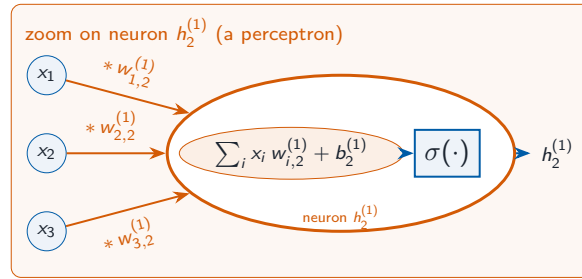
step: 1 if $x > 0$, else 0





Neurons and Matrices (1/5): One Neuron

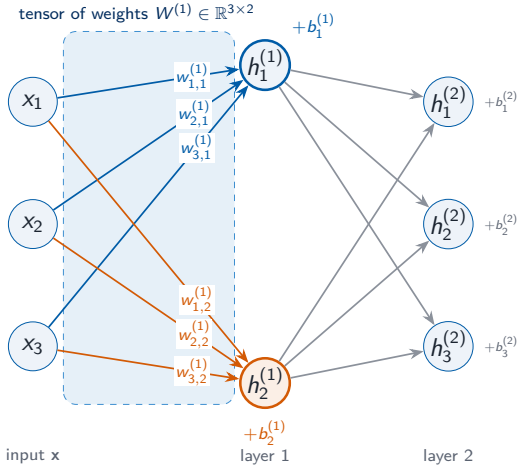




$W \in \mathbb{R}^{N\times M}$: N nodes on the left, M on the right

$$z_2^{(1)} = x_1 w_{1,2}^{(1)} + x_2 w_{2,2}^{(1)} + x_3 w_{3,2}^{(1)} + b_2^{(1)}, \qquad h_2^{(1)} = \sigma\left(z_2^{(1)}\right)$$

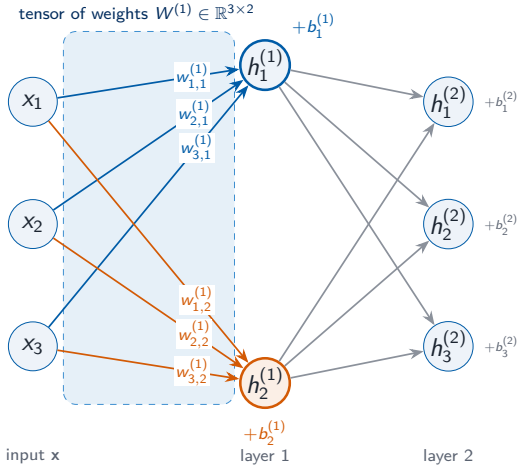
Neurons and Matrices (2/5): Every Neuron



Each neuron adds up what arrives along its edges:

$$z_1^{(1)} = x_1 w_{1,1}^{(1)} + x_2 w_{2,1}^{(1)} + x_3 w_{3,1}^{(1)} + b_1^{(1)}$$

Neurons and Matrices (2/5): Every Neuron

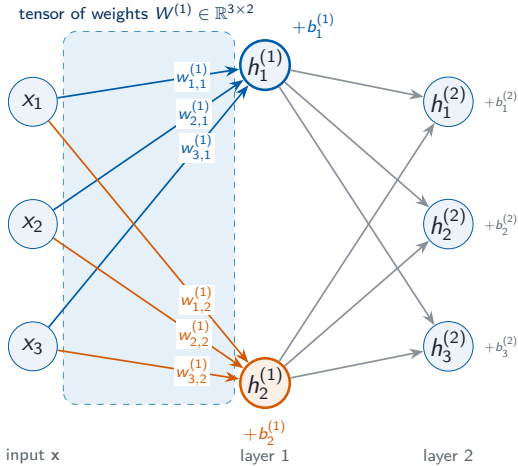


Each neuron adds up what arrives along its edges:

$$z_1^{(1)} = x_1 w_{1,1}^{(1)} + x_2 w_{2,1}^{(1)} + x_3 w_{3,1}^{(1)} + b_1^{(1)}$$

$$z_2^{(1)} = x_1 w_{1,2}^{(1)} + x_2 w_{2,2}^{(1)} + x_3 w_{3,2}^{(1)} + b_2^{(1)}$$

Neurons and Matrices (2/5): Every Neuron



Each neuron adds up what arrives along its edges:

$$z_1^{(1)} = x_1 w_{1,1}^{(1)} + x_2 w_{2,1}^{(1)} + x_3 w_{3,1}^{(1)} + b_1^{(1)}$$

$$z_2^{(1)} = x_1 w_{1,2}^{(1)} + x_2 w_{2,2}^{(1)} + x_3 w_{3,2}^{(1)} + b_2^{(1)}$$

Same pattern for every neuron j — a sum over the inputs i:

$$z_j^{(1)} = \sum_{i=1}^{3} x_i \, w_{i,j}^{(1)} + b_j^{(1)}, \qquad h_j^{(1)} = \sigma\left(z_j^{(1)}\right)$$

Neurons and Matrices (3/5): A Matrix Product

The two neurons, term by term:

$$z_1^{(1)} = x_1 w_{1,1}^{(1)} + x_2 w_{2,1}^{(1)} + x_3 w_{3,1}^{(1)} + b_1^{(1)}$$
$$z_2^{(1)} = x_1 w_{1,2}^{(1)} + x_2 w_{2,2}^{(1)} + x_3 w_{3,2}^{(1)} + b_2^{(1)}$$

Neurons and Matrices (3/5): A Matrix Product

The two neurons, term by term:

$$z_1^{(1)} = x_1 w_{1,1}^{(1)} + x_2 w_{2,1}^{(1)} + x_3 w_{3,1}^{(1)} + b_1^{(1)}$$
$$z_2^{(1)} = x_1 w_{1,2}^{(1)} + x_2 w_{2,2}^{(1)} + x_3 w_{3,2}^{(1)} + b_2^{(1)}$$

Matrix product, by definition

row vector $\mathbf{x}$ times matrix W:

$$(\mathbf{x}\, W)_j = \sum_i x_i\, W_{i,j}$$

= row $\mathbf{x}$ "times" column j of W

Neurons and Matrices (3/5): A Matrix Product

The two neurons, term by term:

$$z_1^{(1)} = x_1 w_{1,1}^{(1)} + x_2 w_{2,1}^{(1)} + x_3 w_{3,1}^{(1)} + b_1^{(1)}$$
$$z_2^{(1)} = x_1 w_{1,2}^{(1)} + x_2 w_{2,2}^{(1)} + x_3 w_{3,2}^{(1)} + b_2^{(1)}$$

Matrix product, by definition

row vector $\mathbf{x}$ times matrix W:

$$(\mathbf{x}\,W)_j = \sum_i x_i\, W_{i,j}$$

= row $\mathbf{x}$ "times" column j of W

This is exactly the sum each neuron computes. All neurons at once:

$$\begin{bmatrix} z_1^{(1)} & z_2^{(1)} \end{bmatrix} = \begin{bmatrix} x_1 & x_2 & x_3 \end{bmatrix} \begin{bmatrix} w_{1,1}^{(1)} & w_{1,2}^{(1)} \\ w_{2,1}^{(1)} & w_{2,2}^{(1)} \\ w_{3,1}^{(1)} & w_{3,2}^{(1)} \end{bmatrix} + \begin{bmatrix} b_1^{(1)} & b_2^{(1)} \end{bmatrix}$$

Neurons and Matrices (3/5): A Matrix Product

The two neurons, term by term:

$$z_1^{(1)} = x_1 w_{1,1}^{(1)} + x_2 w_{2,1}^{(1)} + x_3 w_{3,1}^{(1)} + b_1^{(1)}$$
$$z_2^{(1)} = x_1 w_{1,2}^{(1)} + x_2 w_{2,2}^{(1)} + x_3 w_{3,2}^{(1)} + b_2^{(1)}$$

Matrix product, by definition

row vector $\mathbf{x}$ times matrix W:

$$(\mathbf{x}\,W)_j = \sum_i x_i\, W_{i,j}$$

= row $\mathbf{x}$ "times" column j of W

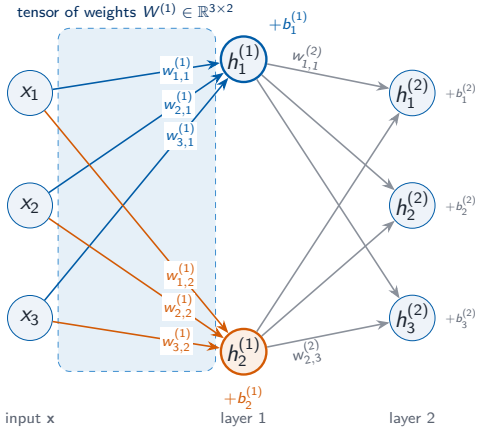
This is exactly the sum each neuron computes. All neurons at once:

$$\begin{bmatrix} z_1^{(1)} & z_2^{(1)} \end{bmatrix} = \begin{bmatrix} x_1 & x_2 & x_3 \end{bmatrix} \begin{bmatrix} w_{1,1}^{(1)} & w_{1,2}^{(1)} \\ w_{2,1}^{(1)} & w_{2,2}^{(1)} \\ w_{3,1}^{(1)} & w_{3,2}^{(1)} \end{bmatrix} + \begin{bmatrix} b_1^{(1)} & b_2^{(1)} \end{bmatrix}$$

$$\boxed{\mathbf{z}^{(1)} = \mathbf{x}\,W^{(1)} + \mathbf{b}^{(1)}}, \qquad \mathbf{h}^{(1)} = \sigma\big(\mathbf{z}^{(1)}\big) \text{ element by element}$$

column j of $W^{(1)}$ = the incoming edges of neuron j; shape (inputs $\times$ neurons) $= 3 \times 2$

Neurons and Matrices (4/5): Layer after Layer



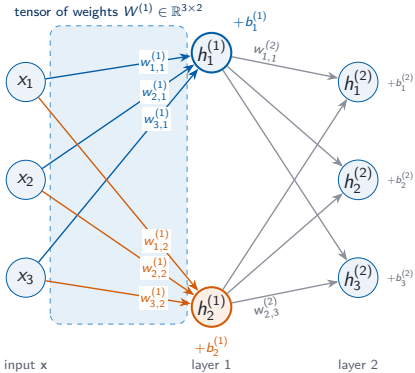
Layer 2 does the same, with $\mathbf{h}^{(1)}$ as its input:

$$h_k^{(2)} = \sigma\Big(\sum_{j=1}^{2} h_j^{(1)}\, w_{j,k}^{(2)} + b_k^{(2)}\Big)$$

$$\iff \quad \mathbf{h}^{(2)} = \sigma\big(\mathbf{h}^{(1)} W^{(2)} + \mathbf{b}^{(2)}\big)$$

$W^{(2)}$: 2×3 (2 inputs, 3 neurons)

Neurons and Matrices (5/5): Any Layer

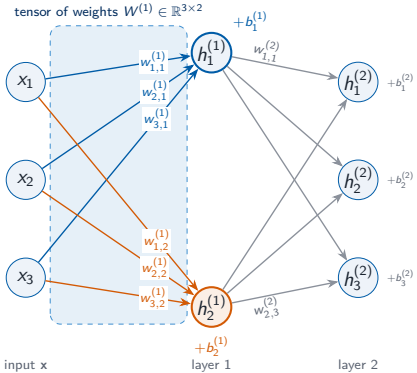


Any layer l $\quad (\mathbf{h}^{(0)} = \mathbf{x})$

$$h_j^{(l)} = \sigma\Big(\sum_i h_i^{(l-1)}\, w_{i,j}^{(l)} + b_j^{(l)}\Big)$$

$$\iff \quad \mathbf{h}^{(l)} = \sigma\big(\mathbf{h}^{(l-1)} W^{(l)} + \mathbf{b}^{(l)}\big)$$

Neurons and Matrices (5/5): Any Layer



Any layer l $\quad (\mathbf{h}^{(0)} = \mathbf{x})$

$$h_j^{(l)} = \sigma\Big(\sum_i h_i^{(l-1)}\, w_{i,j}^{(l)} + b_j^{(l)}\Big)$$

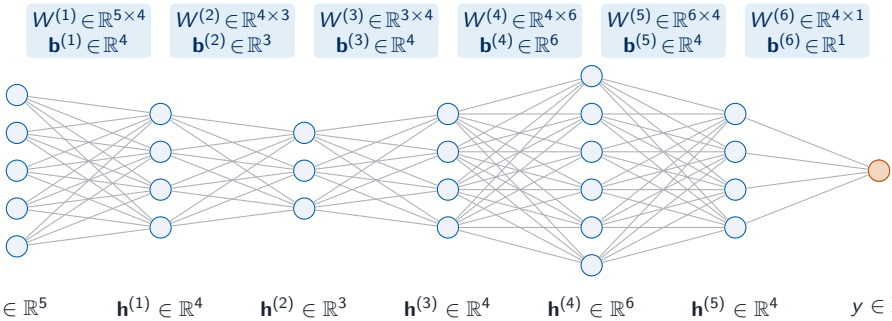
$$\iff \quad \mathbf{h}^{(l)} = \sigma\big(\mathbf{h}^{(l-1)} W^{(l)} + \mathbf{b}^{(l)}\big)$$

A network = nested matrix products and activations:

$$\mathbf{h}^{(2)} = \sigma\Big(\sigma\big(\mathbf{x}W^{(1)} + \mathbf{b}^{(1)}\big)W^{(2)} + \mathbf{b}^{(2)}\Big)$$

N inputs stacked as rows of X: $\sigma(XW^{(1)} + \mathbf{b}^{(1)})$ — one product for the whole batch (as in the MNIST example).

A Deep Network: $f : \mathbb{R}^5 \to \mathbb{R}$ with 5 Hidden Layers

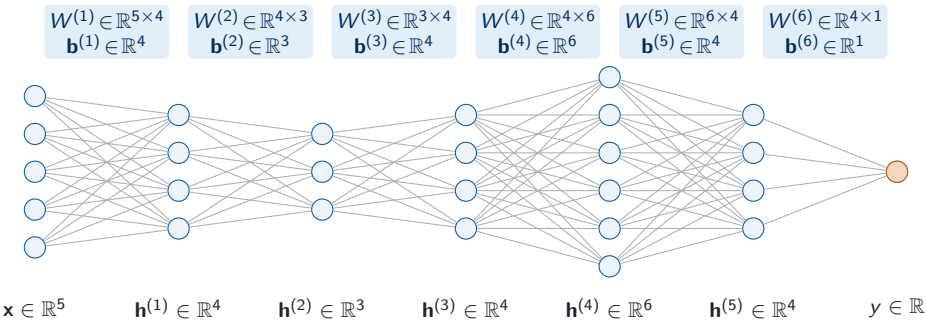


Layer by layer (every hidden layer: matrix product, bias, activation):

$$\mathbf{h}^{(1)} = \sigma\big(\mathbf{x}\, W^{(1)} + \mathbf{b}^{(1)}\big), \quad \mathbf{h}^{(2)} = \sigma\big(\mathbf{h}^{(1)} W^{(2)} + \mathbf{b}^{(2)}\big), \quad \mathbf{h}^{(3)} = \sigma\big(\mathbf{h}^{(2)} W^{(3)} + \mathbf{b}^{(3)}\big),$$
$$\mathbf{h}^{(4)} = \sigma\big(\mathbf{h}^{(3)} W^{(4)} + \mathbf{b}^{(4)}\big), \quad \mathbf{h}^{(5)} = \sigma\big(\mathbf{h}^{(4)} W^{(5)} + \mathbf{b}^{(5)}\big), \quad y = \mathbf{h}^{(5)} W^{(6)} + b^{(6)}$$

the output layer of a regression network is usually linear (no σ)

A Deep Network: $f : \mathbb{R}^5 \to \mathbb{R}$ with 5 Hidden Layers

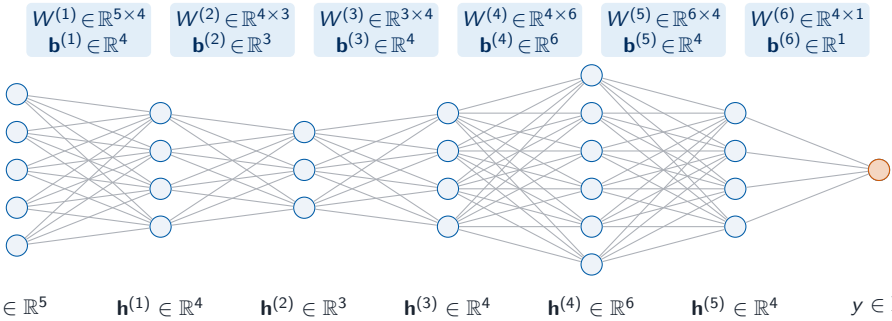


One formula — the argument of each layer is the previous layer:

$$f(\mathbf{x}) = \sigma\Big(\sigma\Big(\sigma\big(\sigma(\mathbf{x}\,W^{(1)} + \mathbf{b}^{(1)})\,W^{(2)} + \mathbf{b}^{(2)}\big)\,W^{(3)} + \mathbf{b}^{(3)}\Big)\,W^{(4)} + \mathbf{b}^{(4)}\Big)\,W^{(5)} + \mathbf{b}^{(5)}\Big)\,W^{(6)} + b^{(6)}$$

read from the inside out: the innermost bracket is the first layer

A Deep Network: $f : \mathbb{R}^5 \to \mathbb{R}$ with 5 Hidden Layers



As a composition of functions: $h^{(l)}(\mathbf{z}) = \sigma(\mathbf{z}\, W^{(l)} + \mathbf{b}^{(l)})$, $h^{(6)}(\mathbf{z}) = \mathbf{z}\, W^{(6)} + b^{(6)}$

$$f = h^{(6)} \circ h^{(5)} \circ h^{(4)} \circ h^{(3)} \circ h^{(2)} \circ h^{(1)}, \qquad f(\mathbf{x}) = h^{(6)}\Big(h^{(5)}\big(\cdots h^{(1)}(\mathbf{x})\cdots\big)\Big)$$

Parameters $\vec{\theta} = \{W^{(l)}, \mathbf{b}^{(l)}\}_{l=1}^{6}$: $24 + 15 + 16 + 30 + 28 + 5 = 118$ numbers.

Universal Approximation Theorem

For a given function $g(x)$, a network with one hidden layer of N neurons,

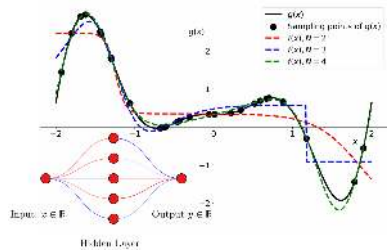
$$f(x; \vec{W}^{(1)}, \vec{b}, \vec{W}^{(2)}) = \sum_{i=1}^{N} W_i^{(2)}\, \sigma(W_i^{(1)} x + b_i),$$

can be made arbitrarily close to g on a compact set K, for a non-polynomial activation σ:

$$\sup_{x \in K} \left| f(x; \vec{W}^{(1)}, \vec{b}, \vec{W}^{(2)}) - g(x) \right| < \epsilon.$$

We find the weights by minimising

$$\mathcal{L}(\vec{\theta}) = \frac{1}{K} \sum_{i=1}^{K} \left| g(x_i) - f(x_i; \vec{\theta}) \right|^2.$$



More hidden neurons $N \to$ better approximation.



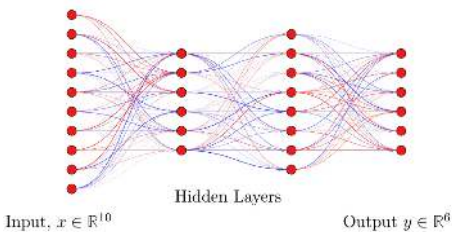
examples of σ; any non-polynomial one works

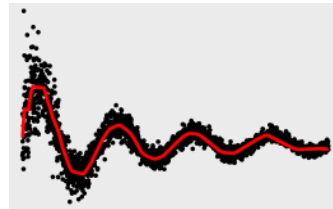
Deep Learning: Non-Linear Curve Fitting

Any continuous function

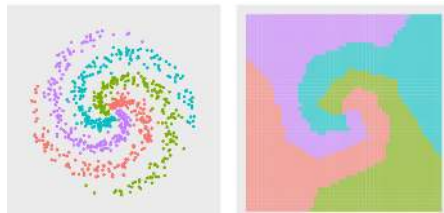
$$f : \mathbb{R}^N \to \mathbb{R}^M$$

can be approximated by a neural network with a single hidden layer.



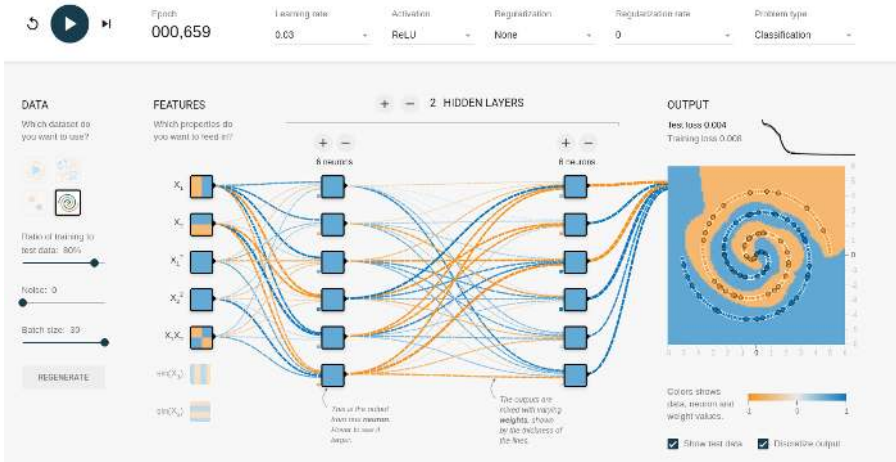


Regression: a curve through noisy data.



Classification: a decision boundary between classes.

Try It Yourself: TensorFlow Playground



playground.tensorflow.org

Elements of Deep Learning

Starting point: a dataset

$$\mathcal{D} = \bigcup_{j=1}^{N_b} \mathcal{D}^{(j)}, \qquad \mathcal{D}^{(j)} = \left\{ \left(\vec{x}_i^{(j)}, \vec{y}_i^{(j)} \right) \right\}, \quad i = 1, \ldots, N_{\mathcal{D}^{(j)}}$$

($\mathcal{D}^{(j)}$: the j-th mini-batch of input–label pairs)

Problem definition: regression or classification?

1. **Loss function:** what do we minimise? $\mathcal{L}^{(j)}(\vec{\theta}_t) = \dfrac{1}{N_{\mathcal{D}^{(j)}}} \displaystyle\sum_{i=1}^{N_{\mathcal{D}^{(j)}}} \mathcal{L}_i\left(\bar{\vec{y}}_i^{(j)}, \vec{y}_i^{(j)} \right)$
2. **Neural network architecture**
3. **Training algorithm**

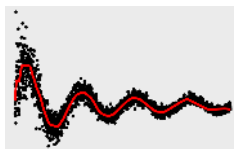
Problem Definition: Regression vs Classification

Regression

predict a **continuous** value

$$y \in \mathbb{R} \quad (\text{or } \vec{y} \in \mathbb{R}^n)$$

e.g. temperature, energy, concentration, binding affinity



output: a number; loss: e.g. mean squared error

Classification

predict a **discrete** label from a given set

$$y \in \{c_1, c_2, \ldots, c_K\}$$

e.g. digit $0 \ldots 9$, cat or dog, ordered or disordered phase



output: probability per class; loss: e.g. cross-entropy

Elements of Deep Learning

Starting point: a dataset

$$\mathcal{D} = \bigcup_{j=1}^{N_b} \mathcal{D}^{(j)}, \qquad \mathcal{D}^{(j)} = \left\{ \left(\vec{x}_i^{(j)}, \vec{y}_i^{(j)} \right) \right\}, \quad i = 1, \ldots, N_{\mathcal{D}^{(j)}}$$

($\mathcal{D}^{(j)}$: the j-th mini-batch of input–label pairs)

Problem definition: regression or classification?

1. **Loss function: what do we minimise?**

$$\mathcal{L}^{(j)}(\vec{\theta}_t) = \frac{1}{N_{\mathcal{D}^{(j)}}} \sum_{i=1}^{N_{\mathcal{D}^{(j)}}} \mathcal{L}_i\left(\vec{\bar{y}}_i^{(j)}, \vec{y}_i^{(j)} \right)$$

2. **Neural network architecture**

3. **Training algorithm**

Loss Function: Regression

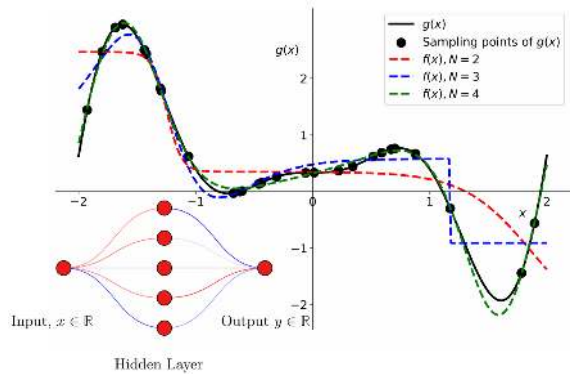
Non-linear curve fitting problem:

$$f_{\vec{\theta}}(\vec{x}) = \vec{\bar{y}} \approx \vec{y}$$

Mean squared error:

$$\mathcal{L}_{\text{MSE}} = \frac{1}{N}\sum_{i=1}^{N}\left(\vec{\bar{y}}_i - \vec{y}_i\right)^2$$

$\vec{y}_i$: true value $\quad \vec{\bar{y}}_i$: network prediction



Encoding Class Labels: One-Hot Vectors

K classes $\rightarrow$ label $=$ vector of length K with a single 1 at the position of the true class:

German Shepherd	$[1, 0]$
Dachshund	$[0, 1]$
digit "0"	$[1, 0, 0, 0, 0, 0, 0, 0, 0, 0]$
digit "2"	$[0, 0, 1, 0, 0, 0, 0, 0, 0, 0]$

The one-hot vector is a **probability distribution** p: all probability on the true class.

Integer labels $0, 1, \ldots, 9$ impose an order

Integers impose an *order* and a *distance*: "9" would be "further" from "1" than "2" is. Classes are categories, not quantities.

In one-hot form every pair of classes is equally far apart.

The network outputs K numbers, normalised to probabilities q (softmax) — the loss compares q with p.

Loss Function: Classification



Class 0

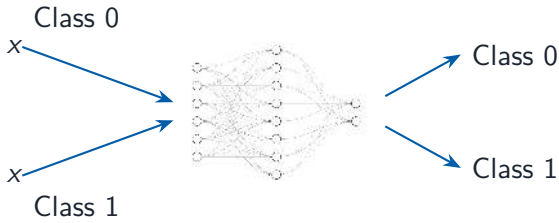


Class 1

Loss Function: Classification







Loss Function: Classification



x

$p(x) = [1, 0]$



x

$p(x) = [0, 1]$



$q(\text{“German Shepherd”} \mid x)$

$q(\text{“Dachshund”} \mid x)$

Loss Function: Classification



$p(x) = [1, 0]$

x



x

$p(x) = [0, 1]$



$q(\text{“German Shepherd”} \mid x)$

$q(\text{“Dachshund”} \mid x)$

p: probability distribution of the *training data*

q: probability distribution given by the *network*

Loss Function: Classification



$p(x) = [1, 0]$

x



x

$p(x) = [0, 1]$



$q(\text{“German Shepherd”} \mid x)$

$q(\text{“Dachshund”} \mid x)$

p: probability distribution of the *training data*

q: probability distribution given by the *network*

How far is q from p?

We need a function of p and q that is 0 for identical distributions and large for very different ones.

Kullback–Leibler Divergence and Cross-Entropy

p: distribution of the training data (one-hot) q: distribution given by the network

Kullback–Leibler divergence:

$$D_{\mathrm{KL}}(p \,\|\, q) = \left\langle \log \frac{p}{q} \right\rangle_p = \sum_c p(c|x) \log \frac{p(c|x)}{q(c|x)}$$

1. $D_{\mathrm{KL}} \geq 0$, and $= 0$ only for $q = p$
2. Asymmetric: $D_{\mathrm{KL}}(p\|q) \neq D_{\mathrm{KL}}(q\|p)$

Kullback–Leibler Divergence and Cross-Entropy

p: distribution of the training data (one-hot) q: distribution given by the network

Kullback–Leibler divergence:

$$D_{\mathrm{KL}}(p \,\|\, q) = \left\langle \log \frac{p}{q} \right\rangle_p = \sum_c p(c|x) \log \frac{p(c|x)}{q(c|x)}$$

1. $D_{\mathrm{KL}} \geq 0$, and $= 0$ only for $q = p$
2. Asymmetric: $D_{\mathrm{KL}}(p\|q) \neq D_{\mathrm{KL}}(q\|p)$

$$D_{\mathrm{KL}}(p\|q) = \underbrace{\sum_c p \log p}_{-S(p)\ \text{fixed by the data}} \underbrace{- \sum_c p \log q}_{\mathcal{L}_{\mathrm{CE}}(p,q)}$$

Only the second term depends on the network $\Rightarrow$ minimise the **categorical cross-entropy** $\mathcal{L}_{\mathrm{CE}}(p, q) = -\sum_c p(c|x) \log q(c|x)$

Elements of Deep Learning

Starting point: a dataset

$$\mathcal{D} = \bigcup_{j=1}^{N_b} \mathcal{D}^{(j)}, \qquad \mathcal{D}^{(j)} = \left\{ \left(\vec{x}_i^{(j)}, \vec{y}_i^{(j)} \right) \right\}, \quad i = 1, \ldots, N_{\mathcal{D}^{(j)}}$$

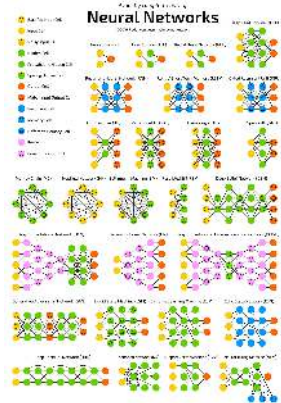
($\mathcal{D}^{(j)}$: the j-th mini-batch of input–label pairs)

Problem definition: regression or classification?

1. **Loss function:** what do we minimise? $\mathcal{L}^{(j)}(\vec{\theta}_t) = \dfrac{1}{N_{\mathcal{D}^{(j)}}} \displaystyle\sum_{i=1}^{N_{\mathcal{D}^{(j)}}} \mathcal{L}_i\left(\bar{\vec{y}}_i^{(j)}, \vec{y}_i^{(j)} \right)$
2. **Neural network architecture**
3. **Training algorithm**

Zoo of Architectures

- **FNN / MLP** — tabular data
- **CNN** — images & spatial data (ResNet, U-Net)
- **RNN / LSTM / GRU** — sequences, time series
- **Transformers** — language, any sequence (GPT, BERT, ViT, LLMs)
- **GNNs** — graphs & molecules
- **Autoencoders / VAEs** — compression
- **Generative models** — *generation* (GANs, diffusion, normalising flows)



Don't memorise the zoo — learn the *principles* that generate it.

Elements of Deep Learning

Starting point: a dataset

$$\mathcal{D} = \bigcup_{j=1}^{N_b} \mathcal{D}^{(j)}, \qquad \mathcal{D}^{(j)} = \left\{ \left(\vec{x}_i^{(j)}, \vec{y}_i^{(j)} \right) \right\}, \quad i = 1, \ldots, N_{\mathcal{D}^{(j)}}$$

($\mathcal{D}^{(j)}$: the j-th mini-batch of input–label pairs)

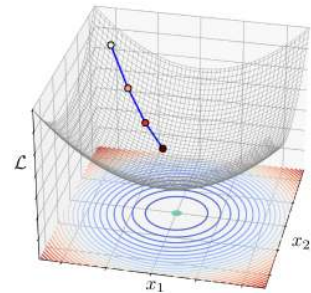
Problem definition: regression or classification?

1. **Loss function:** what do we minimise? $\mathcal{L}^{(j)}(\vec{\theta}_t) = \dfrac{1}{N_{\mathcal{D}^{(j)}}} \displaystyle\sum_{i=1}^{N_{\mathcal{D}^{(j)}}} \mathcal{L}_i\left(\bar{\vec{y}}_i^{(j)}, \vec{y}_i^{(j)} \right)$
2. **Neural network architecture**
3. **Training algorithm**

The Loss Depends on the Data *and* the Parameters

$$\mathcal{L}(\vec{\theta};\, \mathcal{D}) = \frac{1}{N}\sum_{i=1}^{N} \ell\Big(f_{\vec{\theta}}(\vec{x}_i),\, \vec{y}_i\Big), \qquad \mathcal{D} = \{(\vec{x}_i, \vec{y}_i)\}_{i=1}^{N}$$

- $\vec{\theta}$: weights and biases — what we **change**
- $\mathcal{D}$: the training data — **fixed** during training, a finite sample from an unknown distribution
- training: $\vec{\theta}^{*} = \arg\min_{\vec{\theta}} \mathcal{L}(\vec{\theta};\, \mathcal{D})$
- gradients are taken with respect to $\vec{\theta}$, not the data



a loss landscape over the parameters

Different data ⇒ a different landscape: another sample moves the minimum, one reason why training and test performance differ.

Mini-Batches Instead of the Full Dataset

The exact gradient needs *every* training example:

$$\nabla_{\vec{\theta}} \mathcal{L}(\vec{\theta};\, \mathcal{D}) = \frac{1}{N} \sum_{i=1}^{N} \nabla_{\vec{\theta}}\, \ell\big(f_{\vec{\theta}}(\vec{x}_i), \vec{y}_i\big)$$

- **memory:** backpropagation stores the intermediate values of every example; ImageNet (1.28 million images, $>$ 100 GB) does not fit in GPU memory
- **cost:** one update would need a pass over the whole dataset — very few updates per hour

Mini-batch gradient

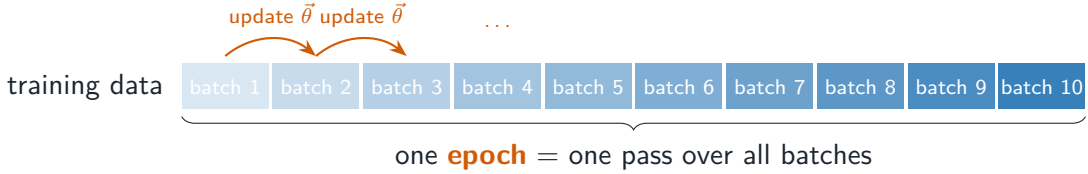
Average over a random subset of $B \ll N$ examples:

$$\nabla_{\vec{\theta}} \mathcal{L} \approx \frac{1}{B} \sum_{i \in \text{batch}} \nabla_{\vec{\theta}}\, \ell_i$$

a noisy but unbiased estimate — and cheap

Lab 009, MNIST, 5 epochs: batch 64 $\rightarrow$ 3910 updates, 93.7% val. accuracy; full batch $\rightarrow$ 5 updates, 36.0%.

Mini-Batches, Iterations, Epochs



One epoch:

1. shuffle the training data
2. split into batches of size B
3. per batch: forward pass $\rightarrow$ loss $\rightarrow$ gradient $\rightarrow$ update $\vec{\theta}$ *(one iteration)*
4. stop after one pass over all examples

Repeat for many epochs; watch training and validation loss.

Example: MNIST

$N = 50\,000$ training images,
$B = 64$
$\Rightarrow \lceil 50\,000/64 \rceil = 782$ iterations per epoch
20 epochs $\Rightarrow$ 15 640 updates of $\vec{\theta}$

Batch size, learning rate and number of epochs are *hyperparameters*, chosen by us.

Parameter Optimisation with Gradient Descent

1. Forward pass

$$\bar{\vec{y}}_i^{(j)} = f_{\vec{\theta}_t}(\vec{x}_i^{(j)})$$

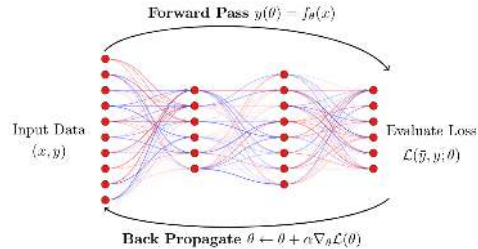
2. Loss calculation

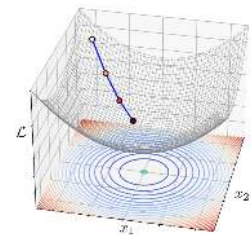
$$\mathcal{L}^{(j)}(\vec{\theta}_t) = \frac{1}{N_{\mathcal{D}^{(j)}}} \sum_{i=1}^{N_{\mathcal{D}^{(j)}}} \mathcal{L}_i(\bar{\vec{y}}_i^{(j)}, \vec{y}_i^{(j)})$$

3. Backpropagation and update

$$\vec{\theta}_{t+1} \leftarrow \vec{\theta}_t - \eta \frac{\partial \mathcal{L}^{(j)}(\vec{\theta}_t)}{\partial \vec{\theta}_t}$$

η: learning rate



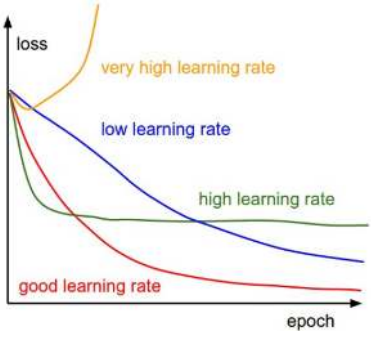


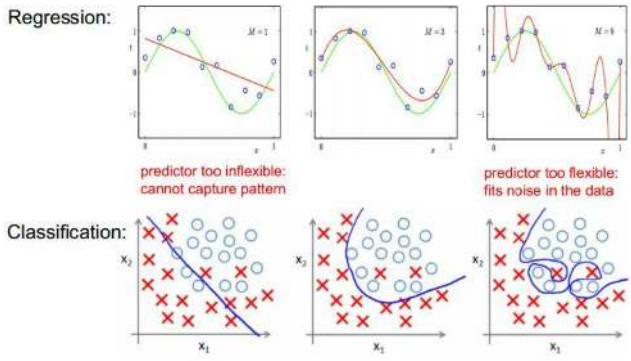
Training Loop

Algorithm 1 Minibatch stochastic gradient descent (SGD)

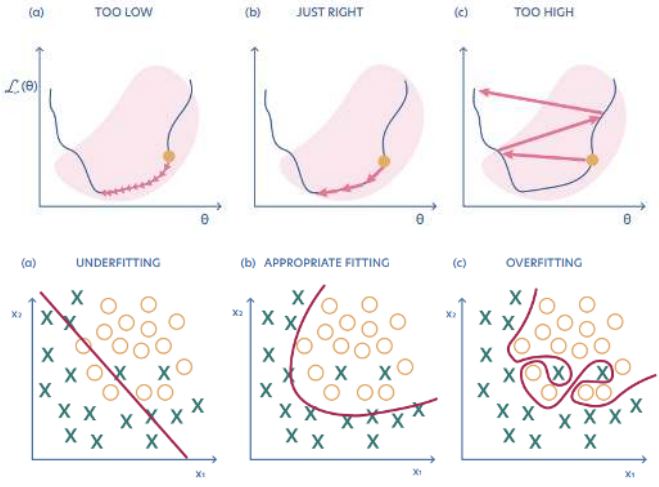
Require: Learning rate η
  Initialize $\boldsymbol{\theta}$ to random values
  for epoch $= 1$ to no_epochs **do**
    Shuffle $\mathcal{D}_{\text{train}}$
    for i $= 1$ to m (where m is a minibatch size) **do**
      $\boldsymbol{x}_i, y_i \sim \mathcal{D}_{\text{train}}$ ▷ Draw random data point from data set without replacement
      $\mathcal{L} \leftarrow \frac{1}{m} \sum_{i=1}^{m} \mathcal{L}(y_i, f(\boldsymbol{x}_i))$ ▷ Compute loss function on the minibatch
      $(\nabla \mathcal{L})_j \leftarrow \frac{\partial \mathcal{L}}{\partial \theta_j}$ ▷ Compute gradients
      $\theta_j \leftarrow \theta_j - \eta \frac{\partial}{\partial \theta_j} \mathcal{L}$ ▷ Update parameters
    end for
  end for
  return $\boldsymbol{\theta}$

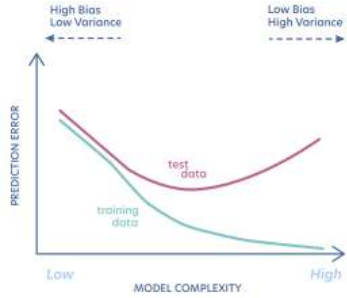
A Well-Trained Network: Learning Rate and Fit





A Well-Trained Network: Underfitting, Overfitting, Bias–Variance





The reason neural networks work so well

Neural Networks Learn the Probability Distribution of Training Data

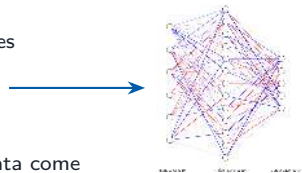
Data:
– labelled images
– text
– video
– tabular data

"Real-world" data come from some "real"/"given" probability distribution



Neural Networks Learn the Probability Distribution of Training Data

Data:
– labelled images
– text
– video
– tabular data



Probability distribution of:
– the label, for classification
– the next word, for text generation
– a set of pixels, for image generation
– the next video frame, for video generation
(spatial and temporal structure)

"Real-world" data come from some "real"/"given" probability distribution

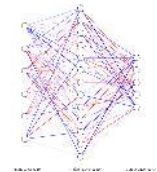


1. The "predicted" distribution depends on the trainable parameters
2. Compare it with the true distribution — **calculate the loss**

NN parameters update

Neural Networks Learn the Probability Distribution of Training Data

Data:
– labelled images
– text
– video
– tabular data



Probability distribution of:
– the label, for classification
– the next word, for text generation
– a set of pixels, for image generation
– the next video frame, for video generation
(spatial and temporal structure)

"Real-world" data come from some "real"/"given" probability distribution



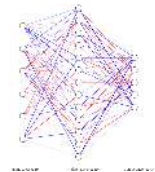
1. The "predicted" distribution depends on the trainable parameters
2. Compare it with the true distribution — **calculate the loss**

NN parameters update

"Bob likes to eat . . . "

Neural Networks Learn the Probability Distribution of Training Data

Data:
– labelled images
– text
– video
– tabular data



Probability distribution of:
– the label, for classification
– the next word, for text generation
– a set of pixels, for image generation
– the next video frame, for video generation
(spatial and temporal structure)

"Real-world" data come from some "real"/"given" probability distribution



1. The "predicted" distribution depends on the trainable parameters
2. Compare it with the true distribution — **calculate the loss**

NN parameters update

"Bob likes to eat . . . "
"Bob likes to eat red . . . "

Today's "AI"



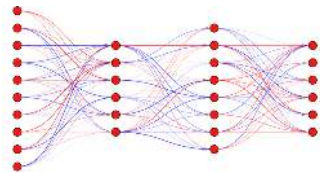
1. **Learning** a probability distribution from the provided data
2. **Sampling** from the learned probability distribution

Deep Learning Now: Data, Compute, Software

Neural networks have been known since the 1950s (perceptron) and trainable since the 1980s (backpropagation). **Why the boom only after ∼2012?**

Enabling factors:

- **Data** — Internet, sensors, digitised archives
- **Compute** — GPUs: built for games, perfect for matrices
- **Software** — Python, automatic differentiation (PyTorch, JAX) computes every gradient
- **Open science** — arXiv, GitHub, shared benchmarks (ImageNet, PDB)



The deep-learning training loop.

Ch. 1 of Heightman & Płodzień, Springer (to appear, 2027), arXiv:2508.10666

Summary of Deep Learning

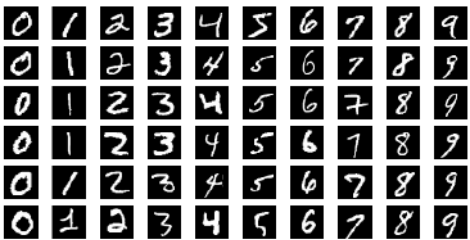
1. Neural networks are **parametrised non-linear functions** with a nested structure
2. Deep learning rests on the **universal approximation theorem**
3. Deep learning is **non-linear curve fitting**
4. Three elements: the **loss** (MSE for regression, cross-entropy for classification), the **architecture**, and the **training algorithm** (gradient descent)
5. Deep learning **learns a probability distribution** from a finite set of samples drawn from a "real", unknown probability distribution

2

Deep Learning “Hello World”: MNIST

JAGIELLONIAN UNIVERSITY
IN KRAKOW

Example #1: MNIST, the *Hello World* of Deep Learning



LeCun et al. (1998)

- hand-written digits $0, \ldots, 9$
- 28×28 grey-scale pixels, values $0 \ldots 255$
- 60 000 training and 10 000 test images
- classes nearly balanced: guessing gives $\approx 10\%$

Splitting the data

training — fit the parameters
validation — choose settings
test — used *once*, for the final quality

Digit Classification: Non-Linear Function Fitting

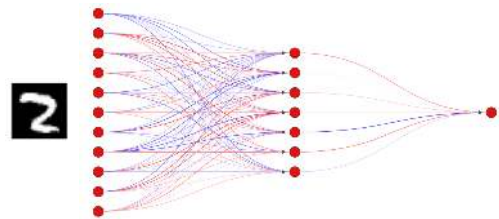
Somewhere there is an unknown function

$$G : \vec{x} \in \mathbb{R}^{784} \to y \in \{0, \ldots, 9\}$$

from pixels to digits — the high-dimensional cousin of $g(x)$ from the universal approximation slide. We only see finitely many samples of it.

The naive model: treat it as **regression**

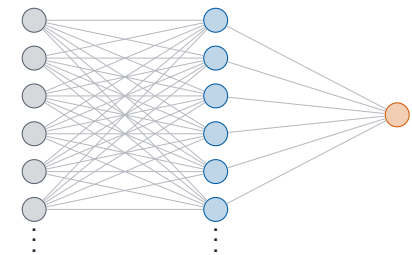
- *one* output node, $\bar{y} = \sigma(\cdot) \in (0, 1)$
- target: the digit divided by 10
- loss: mean squared error
- prediction: $10\,\bar{y}$, rounded



$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

$784 \to 21$ sigmoid nodes $\to 1$ sigmoid output

MNIST Model A: One Output Node



6 of the 784 inputs and 6 of the 21 hidden nodes drawn; bold: true width

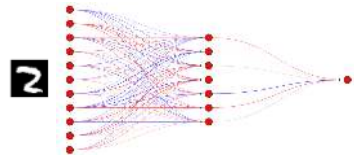
layer	output shape	parameters
input (flattened image)	$(B, 784)$	none
Linear $784 \to 21 + \sigma$	$(B, 21)$	$16\,464 + 21$
Linear $21 \to 1 + \sigma$	$(B, 1)$	$21 + 1$
total		16 507
trainable / frozen		16 507 / 0

B: batch size; σ: sigmoid; $a + b$: weights + biases ($W^{(1)}$: 21×784, $\mathbf{b}^{(1)}$: 21). Lab 009.

Feed-Forward Pass

under the hood

A mini-batch of $N_{\mathcal{D}}$ images, each flattened to $m = 784$ pixels: $X \in \mathbb{R}^{N_{\mathcal{D}} \times m}$



step	operation	shape
1. first linear layer	$\vec{h} = X\vec{W}^{(1)} + \vec{b}^{(1)}$	$[N_{\mathcal{D}}, m] \times [m, N^{(1)}] = [N_{\mathcal{D}}, N^{(1)}]$
2. activation	$\bar{\vec{h}} = \sigma(\vec{h})$	$[N_{\mathcal{D}}, N^{(1)}]$
3. second linear layer	$\vec{g} = \bar{\vec{h}}\,\vec{W}^{(2)}$	$[N_{\mathcal{D}}, N^{(1)}] \times [N^{(1)}, 1] = [N_{\mathcal{D}}, 1]$
4. output activation	$\bar{\vec{y}} = \sigma(\vec{g})$	$[N_{\mathcal{D}}, 1]$

Loss on the mini-batch: $\mathcal{L} = \dfrac{1}{2N_{\mathcal{D}}} \left\| \bar{\vec{y}} - \vec{y} \right\|^2$ ($\vec{y}$: true digits/10)

Parameter Update via the Chain Rule

under the hood

Chain rule, with $\sigma'(z) = \sigma(z)\,(1-\sigma(z))$:

$$\frac{\partial \mathcal{L}}{\partial \vec{W}^{(2)}} = \frac{\partial \mathcal{L}}{\partial \bar{\vec{y}}}\frac{\partial \bar{\vec{y}}}{\partial \vec{g}}\frac{\partial \vec{g}}{\partial \vec{W}^{(2)}}, \qquad \frac{\partial \mathcal{L}}{\partial \vec{W}^{(1)}} = \frac{\partial \mathcal{L}}{\partial \bar{\vec{y}}}\frac{\partial \bar{\vec{y}}}{\partial \vec{g}}\frac{\partial \vec{g}}{\partial \bar{\vec{h}}}\frac{\partial \bar{\vec{h}}}{\partial \vec{h}}\frac{\partial \vec{h}}{\partial \vec{W}^{(1)}}$$

Collect the common factors ($\odot$: element-wise product):

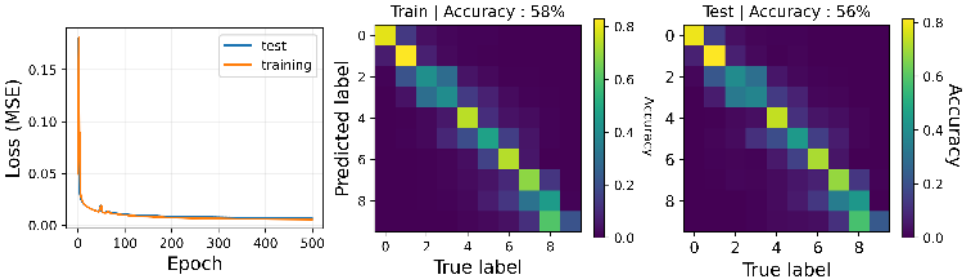
$$Q_2 = (\bar{\vec{y}} - \vec{y}) \odot \bar{\vec{y}} \odot (1 - \bar{\vec{y}}), \qquad Q_1 = (Q_2\, \vec{W}^{(2)\,T}) \odot \bar{\vec{h}} \odot (1 - \bar{\vec{h}})$$

Update rule:

$$\vec{W}^{(2)} \leftarrow \vec{W}^{(2)} - \frac{\eta}{N_{\mathcal{D}}}\,\bar{\vec{h}}^{\,T} Q_2, \qquad \vec{W}^{(1)} \leftarrow \vec{W}^{(1)} - \frac{\eta}{N_{\mathcal{D}}}\,X^T Q_1, \qquad \vec{b}^{(1)} \leftarrow \vec{b}^{(1)} - \frac{\eta}{N_{\mathcal{D}}} \sum_{\text{batch}} Q_1$$

Even this tiny network needs substantial bookkeeping — which is why we let automatic differentiation do it (PyTorch, JAX).

Single Output and MSE: the Result

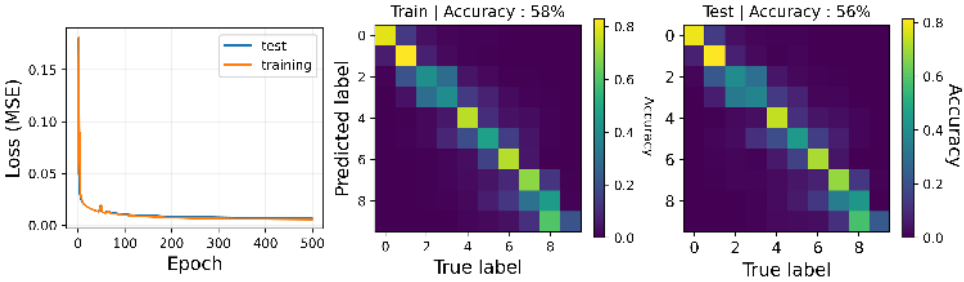


Confusion matrix: row = predicted, column = true digit; accuracy = mean of the diagonal.

Single output node, MSE: about 58% training, 56% test accuracy.

Better than guessing (10%) — but why so poor?

Single Output and MSE: the Result



Confusion matrix: row = predicted, column = true digit; accuracy = mean of the diagonal.

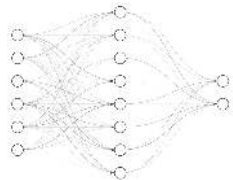
Single output node, MSE: about 58% training, 56% test accuracy.

Better than guessing (10%) — but why so poor?

One number on a line imposes an *ordering*: "3" lies between "2" and "4", and confusing 1 with 9 costs more than 1 with 2. Digits are *categories*, not quantities.

Loss Function: Categorical Cross-Entropy





ten output nodes $z_0, \ldots, z_9$

$p = [0, 0, 1, 0, 0, 0, 0, 0, 0, 0]$ (one-hot label)

$q = [0.1, 0, 0.7, 0.01, 0.1, 0.02, 0, 0.02, 0, 0.05]$

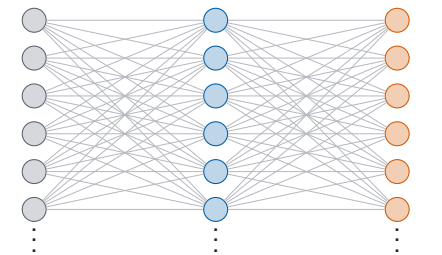
Normalisation (**softmax**): $q_k = q(k \mid x) = \dfrac{e^{z_k}}{\sum_{j=0}^{9} e^{z_j}}$

From the KL divergence (previous section):

$$\mathcal{L}_{\mathrm{CE}}(p, q) = -\sum_{k=0}^{9} p_k \log q_k = -\log q_{\text{true digit}}$$

- no ordering between classes
- network outputs a *probability* for each digit
- untrained network: $\mathcal{L}_{\mathrm{CE}} \approx \ln 10 \approx 2.3$

MNIST Model B: Ten Output Nodes

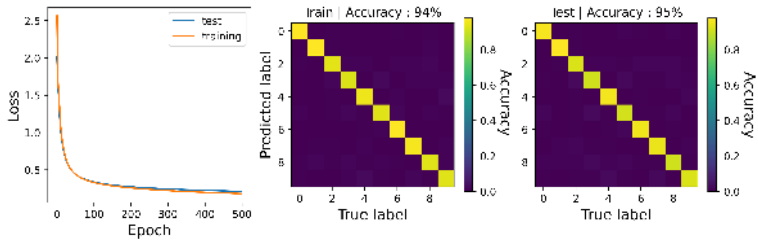


6 of the 784 inputs and 6 of the 21 hidden nodes drawn; bold: true width

layer	output shape	parameters
input (flattened image)	$(B, 784)$	none
Linear $784 \to 21 + \sigma$	$(B, 21)$	$16\,464 + 21$
Linear $21 \to 10$ (logits)	$(B, 10)$	$210 + 10$
total		16 705
trainable / frozen		16 705 / 0

B: batch size (64 in the notebook's graph); σ: sigmoid; z_k: logit of digit k; the softmax sits in the loss, not in the model. Lab 009.

Categorical Cross-Entropy: Result



Same hidden layer (21 sigmoid nodes), ten outputs, cross-entropy: about 94% training, 95% test accuracy.

Adam, mini-batches of 64, 20 epochs	test accuracy
1 output, MSE	65.2%
10 outputs, MSE on one-hot labels	93.8%
10 outputs, cross-entropy	94.8%

Lesson: the *representation of the output* and the *loss* matter as much as the network.

Thank You

Questions, project ideas, feedback: marcin.plodzien@uj.edu.pl

Website · GitHub · Google Scholar



